SOFTWARE DEVELOPMENT LIFE CYCLE

Phases, Teams, Documents, Tools, Roles & Recovery from Failures

A complete walkthrough of how software gets built, tested, shipped and supported

What is SDLC?

A structured process that turns an idea into a working, maintained product

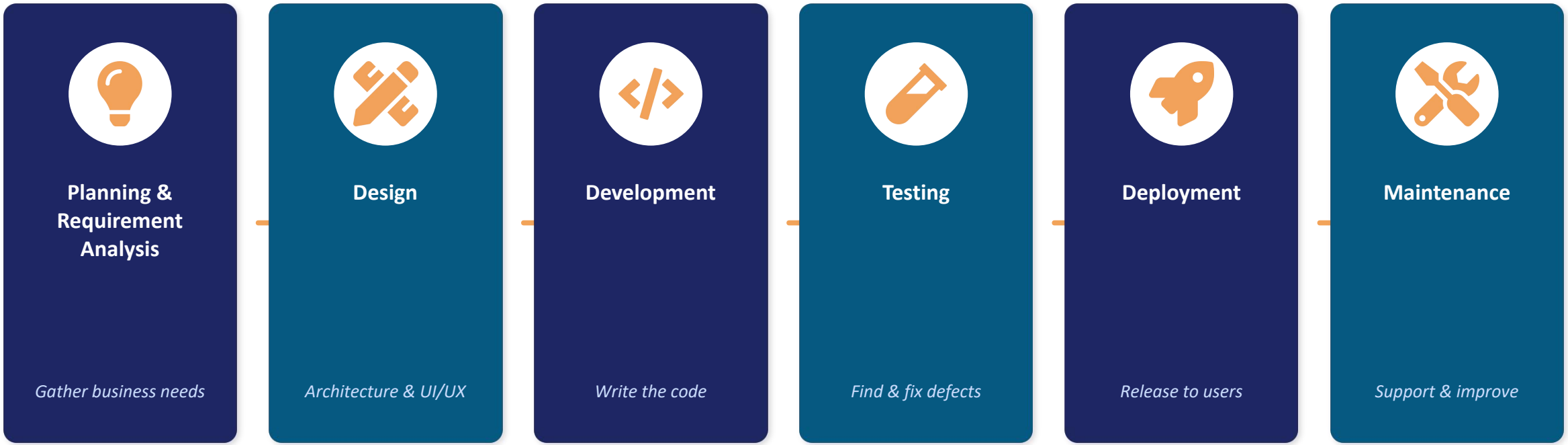
The Software Development Life Cycle (SDLC) is a sequence of phases that software passes through — from gathering requirements to long-term maintenance. Each phase has a clear goal, a responsible team, defined inputs/outputs (documents), and a handoff to the next phase.

- 6 core phases, each with its own owner team
- Documents act as the 'contract' passed between teams
- Different methodologies (Waterfall, Agile, DevOps) run these phases differently
- Most real failures happen at the handoff between teams, not within one phase

1. **Planning**
2. **Design**
3. **Development**
4. **Testing**
5. **Deployment**
6. **Maintenance**

The 6 Phases of SDLC

Each phase flows into the next, forming a continuous cycle



Maintenance feeds back into Planning for the next release — SDLC is a continuous loop, not a one-time line.

Phase 1 — Planning & Requirement Analysis

Owning Team: Business Analysis & Product Team

Roles Involved

- Product Manager
- Business Analyst
- Project Manager
- Client/Stakeholder

Key Documents

- Business Requirement Document (BRD)
- Software Requirement Specification (SRS)
- Feasibility Study Report
- Project Charter

Tools & Technologies

- Jira / Trello
- Confluence
- MS Project
- Miro (for workshops)

Handoff → *SRS is handed to the Design team as the single source of truth for scope.*

Phase 2 — Design

Owning Team: Architecture & UI/UX Team

Roles Involved

- Solution/System Architect
- UI/UX Designer
- Database Designer
- Tech Lead

Key Documents

- High-Level Design (HLD)
- Low-Level Design (LLD)
- Wireframes & Prototypes
- Database/ER Diagrams

Tools & Technologies

- Figma / Adobe XD
- Lucidchart / draw.io
- ER/Studio
- Enterprise Architect

Handoff → *HLD/LLD + UI mockups are handed to Developers as the build blueprint.*

Phase 3 — Development

Owning Team: Engineering / Development Team

Roles Involved

- Frontend Developer
- Backend Developer
- Full-Stack Developer
- Tech Lead / Code Reviewer

Key Documents

- Coding Standards Guide
- API Documentation
- Source Code Repository
- Sprint/Backlog Tickets

Tools & Technologies

- Git/GitHub/GitLab
- VS Code / IntelliJ
- Node.js, Java, Python, .NET
- Jenkins/CI for builds

Handoff → *Built features + API docs are handed to QA for verification.*

Phase 4 — Testing

Owning Team: Quality Assurance (QA) Team

Roles Involved

- QA / Test Engineer
- Automation Engineer
- Performance Tester
- Security Tester

Key Documents

- Test Plan
- Test Cases & Scripts
- Bug/Defect Reports
- Test Summary Report

Tools & Technologies

- Selenium / Cypress
- JIRA / Zephyr (defect tracking)
- Postman (API testing)
- JMeter (load testing)

Handoff → *Sign-off report and verified build are handed to the DevOps/Release team.*

Phase 5 — Deployment

Owning Team: DevOps & Release Team

Roles Involved

- DevOps Engineer
- Release Manager
- Site Reliability Engineer (SRE)
- Cloud Engineer

Key Documents

- Deployment/Release Plan
- Rollback Plan
- Release Notes
- Infrastructure-as-Code scripts

Tools & Technologies

- Docker / Kubernetes
- AWS / Azure / GCP
- Jenkins / GitHub Actions
- Terraform / Ansible

Handoff → *Live system + monitoring dashboards are handed to the Support/Maintenance team.*

Phase 6 — Maintenance & Support

Owning Team: Support & Maintenance Team

Roles Involved

- Production Support Engineer
- Customer Support
- Maintenance Developer
- Site Reliability Engineer

Key Documents

- Incident/Ticket Logs
- Change Request (CR) Forms
- System Runbooks
- Service Level Agreement (SLA)

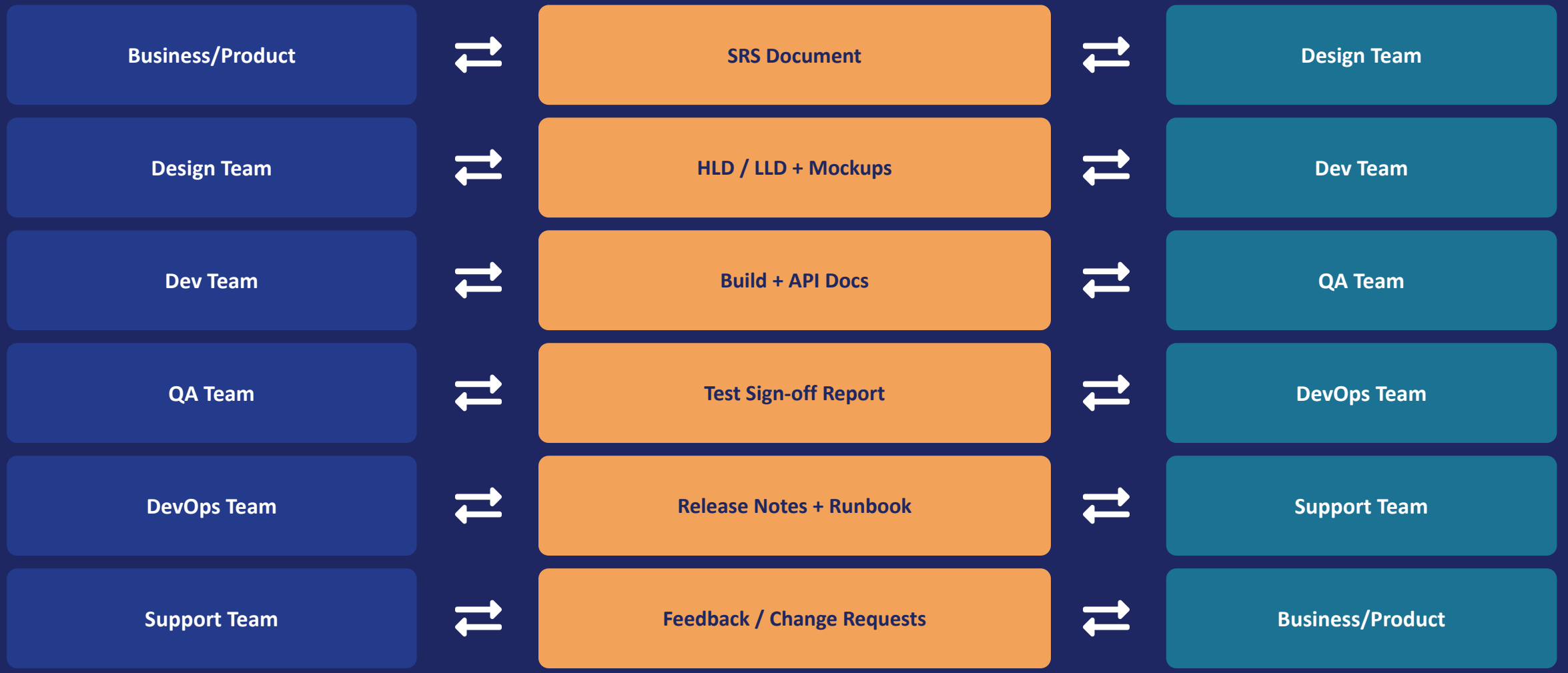
Tools & Technologies

- ServiceNow / Zendesk
- Datadog / New Relic (monitoring)
- PagerDuty (alerting)
- Splunk / ELK (logs)

Handoff → *Feedback & change requests loop back into Planning — restarting the cycle.*

How Teams Interchange & Collaborate

Documents and tools are the bridges between teams across the cycle



Common Problems & How to Recover

Issues that typically occur in each phase, and the practical fix

Planning

Problem: Unclear or changing requirements

Recovery: Lock scope with a signed-off SRS; use change-control process for new asks

Design

Problem: Architecture doesn't scale / wrong tech choice

Recovery: Conduct design reviews & proof-of-concepts before full build-out

Development

Problem: Bugs, technical debt, missed deadlines

Recovery: Code reviews, pair programming, CI pipelines, refactoring sprints

Testing

Problem: Late-found defects, flaky/insufficient tests

Recovery: Shift-left testing, automated regression suites, clear bug triage

Deployment

Problem: Failed release, downtime, config errors

Recovery: Blue-green/canary deploys, automated rollback plan, staging rehearsal

Maintenance

Problem: Production incidents, slow support response

Recovery: 24/7 monitoring & alerting, runbooks, root-cause-analysis postmortems

Methodologies: How Phases Are Run

The same 6 phases, organized differently depending on approach

Waterfall

Phases run strictly in sequence, one completed before the next starts. Good for fixed, well-understood requirements.

Agile / Scrum

Phases repeat in short sprints (1-4 weeks); teams continuously plan, build, test and release in cycles.

DevOps

Development and Operations merge; CI/CD pipelines automate build, test and deployment for continuous delivery.

Regardless of methodology, the same documents, roles and tools shown earlier apply within each phase.

Key Takeaways

- SDLC has 6 phases: Planning, Design, Development, Testing, Deployment, Maintenance
- Each phase has its own team, roles, documents, and tools
- Documents (SRS, HLD/LLD, Test Reports, Release Notes) are the handoff bridge between teams
- Most failures happen at handoffs — clear documentation and communication prevent this
- Agile/DevOps run these phases in fast, repeating cycles rather than once, sequentially

Thank You